

Serialization

- This code demonstrates how to serialize a Java object and save it to a file.

👉 The Student Class

```
import java.io.*;

class Student implements Serializable {
    private int id;
    private String name;
    private int age;

    // Constructor to initialize student data
    public Student(int id, String name, int age) {
        this.age = age;
        this.id = id;
        this.name = name;
    }

    // Method to display student information
    public void disp() {
        System.out.println("id " + id);
        System.out.println("name " + name);
        System.out.println("age " + age);
    }
}
```

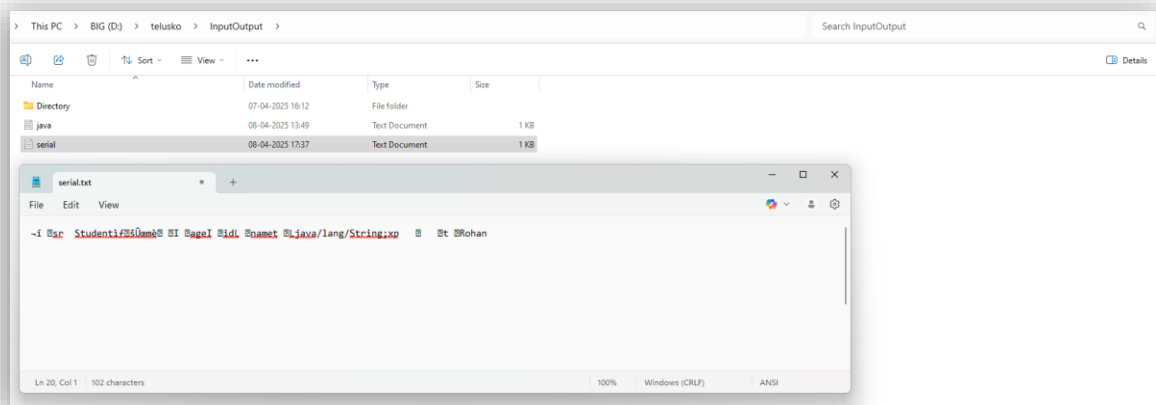
- The class implements `Serializable` interface, which marks it as eligible for serialization.

👉 The Main Class (LaunchIO)

```
public class LaunchIO {  
    public static void main(String[] args) {  
        // Create a Student object  
        Student stu1 = new Student(1, "Rohan", 16);  
        stu1.disp(); // Display object details before serialization  
  
        try {  
            // Create a file output stream to the destination file  
            FileOutputStream fos = new FileOutputStream("D:\\telusko\\InputOutput\\serial.txt");  
  
            // Create an object output stream connected to the file stream  
            ObjectOutputStream oos = new ObjectOutputStream(fos);  
  
            // Serialize the object and write it to the file  
            oos.writeObject(stu1);  
  
            System.out.println("Please check the file to see serialized object");  
  
            // Close the streams  
            oos.close();  
            fos.close();  
  
        } catch (FileNotFoundException ex) {  
            ex.printStackTrace();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Output:

```
id 1  
name Rohan  
age 16  
Please check the file to see serialized object
```



👉 Explanation

➤ Object Creation:

- We create a `Student` object (`stu1`) with ID 1, name "Rohan", and age 16.
- The `disp()` method shows the object's data before serialization.

➤ Stream Setup:

- `FileOutputStream` connects to the physical file where we want to store our serialized object.
- `ObjectOutputStream` wraps the file stream to handle the conversion of Java objects to bytes.

➤ Serialization:

- The `writeObject(stu1)` method converts our `Student` object into a stream of bytes and writes them to the file.
- If you open the file, the content will appear as non-readable characters (binary data).

➤ Resource Cleanup:

- We close both streams to release system resources.

➤ Exception Handling:

- `FileNotFoundException`: Handles cases where the specified file path is invalid.
- `IOException`: Handles other I/O errors that might occur during serialization.

👉 Remember

- The serialized object is stored in binary format, making it secure and not human-readable.
- Once serialized, you can later deserialize it to recreate the original object.
- All instance variables of the object are serialized by default.
- The `Serializable` interface is necessary - without it, you'll get a `NotSerializableException`.
- If a class has references to other objects, those classes must also implement `Serializable`.